

Neural networks

Instructor: Roberto Colomboni

version of 28 September 2026

In regression and classification, we chose a family of predictors and then fitted its parameters by minimising a loss. In Lecture 2, we also saw how to construct nonlinear predictors using feature maps such as $\varphi(x) = x^2$. There, we chose the feature functions *before* fitting the model, and then used the data to determine the coefficients that combined their values. The fitting procedure could therefore change the contribution of each feature, but it could not change the feature functions themselves.

We now ask whether we can also use the data to adjust those functions. To make this possible, we will give the feature functions their own parameters and fit them together with the coefficients of the final prediction. Neural networks provide a way to organise this construction through successive affine transformations and simple nonlinear functions.

We will first follow these calculations step by step to understand how a network produces a prediction. We will then construct two examples to see what the hidden features make possible. Finally, we will return to the loss and explain how the fitting procedure adjusts all the parameters of the network.

1 Changing the predictor while keeping the learning problem

1.1 What we keep from regression and classification

In the previous lectures, we constructed predictors by choosing features and combining their values with adjustable coefficients. The features could be the original input coordinates, or nonlinear transformations chosen before fitting the model. In both cases, the fitting procedure adjusted the coefficients used to combine the feature values, while leaving the feature functions themselves fixed.

When we used the original input coordinates as features, a single output took the affine form

$$f_{\theta}(\mathbf{x}) = \mathbf{x}\mathbf{w} + b, \quad \theta = \begin{pmatrix} b \\ \mathbf{w} \end{pmatrix}, \quad (1)$$

where the input $\mathbf{x}$ is a row, $\mathbf{w}$ is a column of feature weights, and b is the intercept. The parameter column θ collects all the adjustable coefficients. Using transformed features followed the same principle, but we combined the values of those features instead of the original input coordinates.

The interpretation of the model's output depended on the prediction problem. In regression, we used it directly as a numerical prediction. In binary classification, instead, we interpreted it as a score. The sigmoid then converted the score into a probability, and the plug-in rule converted that probability into a predicted class. For multiclass classification, we used one score per class. The softmax converted these scores into class probabilities, and the plug-in rule selected a class with the highest probability.

These steps will remain the same when we use neural networks. A network will produce a numerical prediction in regression or scores in classification, with the sigmoid or softmax and the plug-in rule applied as before. We will also keep the same losses for fitting the model:

squared loss for regression, logistic loss for binary classification, and cross-entropy loss for multiclass classification.

1.2 From fixed features to learned features

To understand what we want to add, let us recall why we introduced nonlinear features in the first place. An affine function of a single input has a straight-line graph, so it cannot represent a relationship whose slope changes with the input. Similarly, in binary classification with two input features, an affine score gives a straight decision boundary, which may be insufficient to separate the classes.

Transforming the input gave us a way to go beyond these restrictions. For example, in Lecture 2 we considered predictors of the form

$$x \mapsto b + w_1x + w_2x^2. \quad (2)$$

The feature map $x \mapsto x^2$ allowed the prediction to curve as x changed. By adjusting w_1 and w_2 , the fitting procedure could change the slope and curvature of the prediction. However, it could not change our choice of the feature functions x and x^2 .

We now ask whether we can also use the data to adjust the feature functions. To make this possible, we will construct features that contain their own adjustable weights and intercepts. We will then fit these parameters together with the coefficients that combine the feature values into a prediction. Consequently, the fitting procedure will be able to change both the feature functions and how their values contribute to the prediction.

This is the sense in which a neural network will learn features. We will still choose how to construct the features, including the operations used and the number of intermediate values computed. However, the parameters inside those operations will be adjusted during fitting. This will give us a way to construct nonlinear predictors and scores while allowing the feature transformations themselves to adapt to the data.

To build such a model, we will organise the calculation into successive stages. We will first examine what happens when each stage is affine. As we will see, composing affine transformations still gives an affine function, so nonlinear operations will be needed to obtain nonlinear features.

2 Composing affine transformations

In multiclass classification, we computed several affine scores from the same input, one for each class. We can describe the whole calculation as a single affine transformation with several outputs. This notation will let us examine what happens when we use the output of one affine transformation as the input to another.

Previously, we collected the weights and intercepts in a single parameter matrix. Here, we will write them separately so that we can follow what happens to each of them when we compose the transformations. We recall that the intercepts are also called *biases*, and we stress that they remain adjustable parameters that we fit together with the weights.

Suppose that the input is a row $\mathbf{u} \in \mathbb{R}^{1 \times n}$ and that we want to compute m outputs. We collect the weights in a matrix $\mathbf{W} \in \mathbb{R}^{n \times m}$, with one column for each output, and the biases in a row $\mathbf{b} \in \mathbb{R}^{1 \times m}$. The affine transformation then produces the output row

$$\mathbf{a} = \mathbf{u}\mathbf{W} + \mathbf{b}. \quad (3)$$

To see how this expression collects the individual calculations, consider its j -th coordinate:

$$a_j = \sum_{k=1}^n u_k W_{kj} + b_j, \quad j = 1, \dots, m. \quad (4)$$

The entry W_{kj} multiplies input coordinate u_k when we compute output coordinate a_j . Therefore, the first index identifies an input and the second identifies an output. Each output has its own weights and bias, while all the outputs are computed from the same input row.

For example, an affine transformation with two input coordinates and three output coordinates uses

$$\mathbf{W} = \begin{pmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_1 & b_2 & b_3 \end{pmatrix}. \quad (5)$$

Its three output coordinates are therefore

$$\begin{aligned} a_1 &= u_1 W_{11} + u_2 W_{21} + b_1, \\ a_2 &= u_1 W_{12} + u_2 W_{22} + b_2, \\ a_3 &= u_1 W_{13} + u_2 W_{23} + b_3. \end{aligned} \quad (6)$$

If we needed only one output, $\mathbf{W}$ would have a single column, recovering the weight vector $\mathbf{w}$ used in regression.

Figure 1 shows how we evaluate this transformation. We first compute the weighted sums, represented by the nodes z_1 , z_2 , and z_3 . We then add the corresponding biases to obtain a_1 , a_2 , and a_3 . The diagram separates these operations to show how they contribute to the same affine transformation.

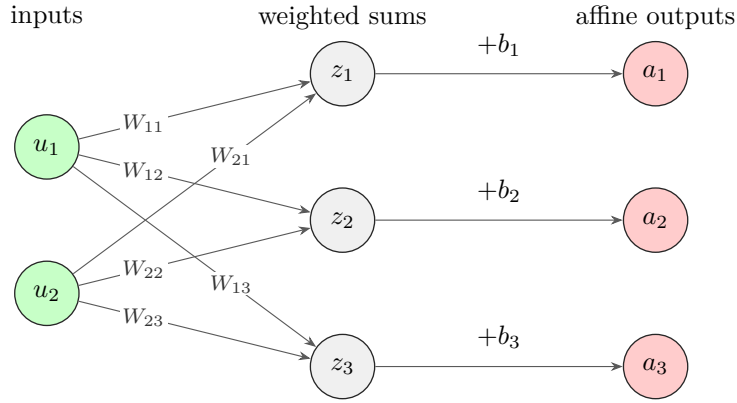


Figure 1: An affine transformation with two input coordinates and three output coordinates. We first form the weighted sums $\mathbf{uW}$, then add the bias row $\mathbf{b}$ to obtain $\mathbf{a} = \mathbf{uW} + \mathbf{b}$. Each circle represents a value, while the labels $+b_j$ on the arrows indicate the bias additions.

2.1 Applying two transformations in succession

We can now ask whether applying one affine transformation after another allows us to represent a non-affine function. Let us first check a simple example with one input and one output. Take $u(x) = 2x + 1$ and $v(z) = 3z - 4$. Applying u first and then using its output as the input to v gives

$$(v \circ u)(x) = 3(2x + 1) - 4 = 6x - 1. \quad (7)$$

The slope and intercept have changed, but the resulting function is still affine. In fact, we could obtain the same output by applying the single affine function $x \mapsto 6x - 1$ directly to the input.

The same conclusion holds for vector inputs and outputs. To keep track of the parameters, we will write $\mathbf{W}_1, \mathbf{b}_1$ for the weights and biases of the first transformation and $\mathbf{W}_2, \mathbf{b}_2$ for those of the second.

Lemma 1 (Composition of affine maps). *The composition of two affine maps is an affine map.*

Proof. Suppose that the first transformation is $f(\mathbf{x}) = \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1$ and the second is $g(\mathbf{z}) = \mathbf{z}\mathbf{W}_2 + \mathbf{b}_2$, with dimensions that allow us to apply g to the output of f . Substituting the expression for $f(\mathbf{x})$ into g and expanding the matrix product gives

$$(g \circ f)(\mathbf{x}) = (\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 = \mathbf{x} \underbrace{(\mathbf{W}_1\mathbf{W}_2)}_{=:\mathbf{W}_{\text{eff}}} + \underbrace{(\mathbf{b}_1\mathbf{W}_2 + \mathbf{b}_2)}_{=:\mathbf{b}_{\text{eff}}}. \quad (8)$$

The matrix $\mathbf{W}_{\text{eff}}$ and the row $\mathbf{b}_{\text{eff}}$ depend only on the parameters of the two transformations. Once those parameters are fixed, the composition therefore multiplies the input by a fixed matrix and adds a fixed row. This is again an affine transformation. $\square$

We can repeat this argument when there are more than two transformations. First, we replace the first two by a single affine transformation. We then compose that transformation with the third, and continue in the same way. Consequently, any finite sequence of affine transformations can be replaced by a single affine transformation.

This means that repeatedly applying affine transformations cannot produce feature functions that are non-affine in the original input. If we use the resulting output as a binary classification score, it also remains an affine score, with the same restriction on the decision boundary.

To go beyond these restrictions, we introduce a nonlinear operation between the affine transformations.

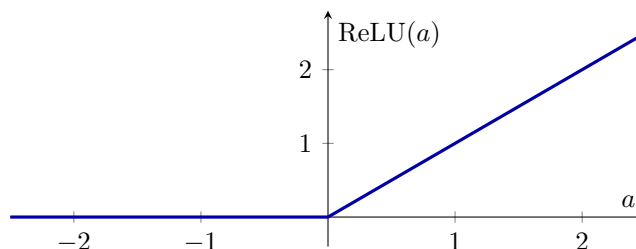
3 Neural networks and the forward pass

3.1 ReLU as a scalar function and as a vector transformation

We will use the *rectified linear unit*, or *ReLU*. For a scalar input, we denote this function by ReLU and define it by

$$\text{ReLU} : \mathbb{R} \longrightarrow \mathbb{R}, \quad \text{ReLU}(a) = \max\{0, a\} = \begin{cases} 0, & a \leq 0, \\ a, & a > 0. \end{cases} \quad (9)$$

ReLU leaves a positive input unchanged and replaces a negative input by zero. Its output is therefore nonnegative. The definition also specifies $\text{ReLU}(0) = 0$. The graph of the ReLU function has a kink at zero, where the function is not differentiable. We will discuss this point in Section 6.



An affine transformation can produce a row of values, so we also need to specify how ReLU acts on such a row. As we did for the sigmoid in Lecture 3, we use bold type to distinguish the vector transformation **ReLU** from the scalar function ReLU. We define the vector transformation by applying the scalar function separately to each coordinate:

$$\mathbf{ReLU}(\mathbf{a}) = \left(\text{ReLU}(a_1) \quad \cdots \quad \text{ReLU}(a_m) \right), \quad \mathbf{a} \in \mathbb{R}^{1 \times m}. \quad (10)$$

Notice that this is a coordinatewise transformation, and it leaves the shape of the row unchanged. For example,

$$\mathbf{ReLU}\left(\begin{pmatrix} -2 & 0 & 3 \end{pmatrix}\right) = \begin{pmatrix} 0 & 0 & 3 \end{pmatrix}.$$

3.2 Reading a neural network diagram

We will build a neural network by combining the operations we introduced above. Suppose that our input is the row $\mathbf{x} = (x_1, x_2)$. With weights w_1, w_2 and a bias b , we can form the affine expression $w_1x_1 + w_2x_2 + b$. Applying ReLU to its value gives

$$\text{ReLU}(w_1x_1 + w_2x_2 + b).$$

Once the weights and bias have been specified, this expression defines a function that assigns one number to each input row. We can therefore use it as a new feature of the input, as discussed in Section 1.

To obtain several features, we can make several copies of this construction, each with its own adjustable weights and bias. For example, five such calculations produce five feature values from the same two input coordinates. We call each of these calculations a *unit*, or also a *neuron*. We group the five units into a *layer*. In the overview diagram below (see Figure 2), each neuron is represented by one circle, and the five circles are arranged in a column (the layer).

These five values can now become the *inputs* to another layer. We can form weighted sums of them, add biases, and apply ReLU again to obtain further features. By continuing this construction and adding a final layer that produces the outputs, we obtain a *neural network*. The layers that produce intermediate features are called *hidden layers*, while the final one is called the *output layer*.

The names *neuron* and *neural network* are inspired by networks of biological neurons, such as those in the brain. A biological neuron receives signals from other neurons and can send signals to others. In our mathematical model, each unit combines incoming values to produce an output, with weights controlling how strongly each input contributes. Connecting these units allows their outputs to become inputs to other units. We stress that this is a loose analogy: biological neurons and their interactions are much more complex.

Figure 2 shows an example. The two circles on the left represent the input coordinates. Each circle after the input represents a neuron, and each connection shows that the value produced

at its starting point is used in the calculation at its endpoint. We number the layers from left to right, without counting the input.

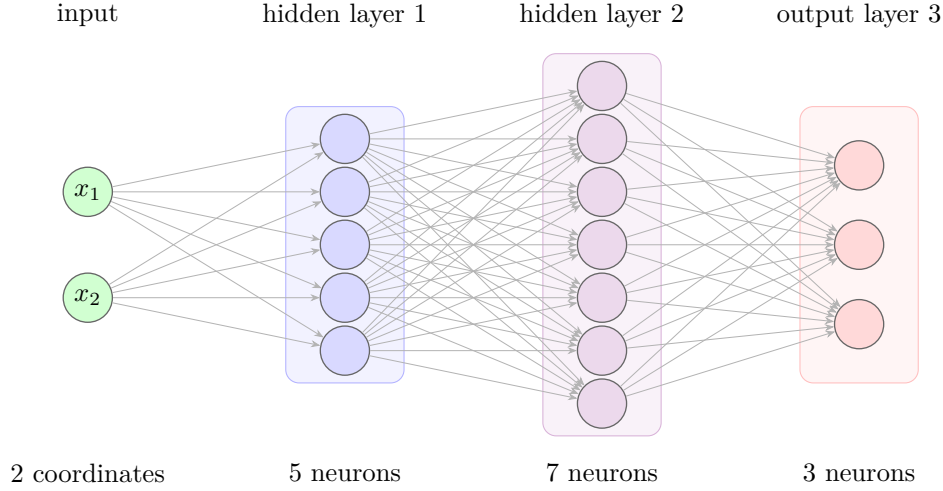


Figure 2: A network with two input coordinates, two hidden layers containing five and seven neurons, respectively, and an output layer with three neurons.

The first highlighted column contains the five feature calculations described above. Each neuron forms an affine combination of the two input coordinates and then applies ReLU, so the layer produces five values. These become the inputs to the second highlighted column. Each of its seven neurons forms a weighted sum of all five incoming values, adds its own bias, and applies ReLU. The second layer therefore produces seven new feature values.

The output layer then combines these seven values to produce three outputs. Each output neuron forms a weighted sum and adds its own bias. In this network, we do *not* apply ReLU in the output layer, so these three affine combinations are the final outputs. Once all the weights and biases have been specified, the whole network is therefore a function from the original two input coordinates to three output values.

The number of units in a layer is called its *width*. Since each unit produces one value, the width also tells us how many values the layer produces. The network in the figure has three layers, with widths five, seven, and three. Evaluating their calculations in order, from the input to the output, is called a *forward pass*.

We will now write these calculations using matrices and rows, as we did for affine transformations. We will first consider a network with just one hidden layer and an output layer, so that we can introduce the notation before extending it to several hidden layers.

3.3 A network with one hidden layer

Let the input row $\mathbf{x}$ have d coordinates, and suppose that the hidden layer has width m and the output layer has width q . The hidden layer will therefore produce m feature values, which the output layer will combine into q outputs.

Each hidden unit needs one weight for each of the d input coordinates and one bias. We collect the weights for the m units in a matrix $\mathbf{W}_1$, with one column per unit, and their biases in a row $\mathbf{b}_1$. Consequently,

$$\mathbf{W}_1 \in \mathbb{R}^{d \times m}, \quad \mathbf{b}_1 \in \mathbb{R}^{1 \times m}.$$

The entry $(\mathbf{W}_1)_{kj}$ is the weight multiplying input coordinate x_k in hidden unit j , while $(\mathbf{b}_1)_j$ is that unit's bias. The subscript 1 on the matrix and bias row identifies the first layer.

Similarly, each output unit needs m weights to combine the hidden values and one bias. Collecting these parameters for the q output units gives

$$\mathbf{W}_2 \in \mathbb{R}^{m \times q}, \quad \mathbf{b}_2 \in \mathbb{R}^{1 \times q}.$$

Here the subscript 2 identifies the output layer, which is the second layer of this network.

The number of layers, their widths, and the functions used inside the units specify the *architecture* of the neural network. In this example, we have chosen one hidden layer that uses ReLU and an output layer that forms affine combinations. The parameters of this network are the entries of $\mathbf{W}_1$, $\mathbf{b}_1$, $\mathbf{W}_2$, and $\mathbf{b}_2$. We collect all these adjustable numbers in a single parameter column, in an order that we keep fixed, and denote it by $\boldsymbol{\theta}$. As in the previous lectures, this will allow us to write the predictor and its loss as functions of the parameters.

We can now evaluate the network for any fixed choice of those parameters. First, we form the weighted sums for all the hidden units and add their biases. This produces a row of m values before ReLU is applied. These values are called *preactivations*, and we denote their row by $\mathbf{a}_1(\mathbf{x})$:

$$\mathbf{a}_1(\mathbf{x}) = \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1. \quad (11)$$

Next, we apply ReLU to each preactivation. The resulting output of a hidden neuron is called its *activation*, and ReLU is therefore referred to as an *activation function*. We denote the row of hidden activations by $\boldsymbol{\phi}_1(\mathbf{x})$.¹ Using **ReLU** for the coordinatewise transformation gives

$$\boldsymbol{\phi}_1(\mathbf{x}) = \text{ReLU}(\mathbf{a}_1(\mathbf{x})). \quad (12)$$

This row still has m entries, because applying ReLU changes the values without changing the number of coordinates. The function $\boldsymbol{\phi}_1$ is therefore a *feature map*, playing the same role as $\boldsymbol{\phi}$ in Lecture 2: it assigns a row of feature values to each input. Here, however, the feature functions depend on the adjustable weights and biases of the hidden layer. We write $\boldsymbol{\phi}_1$ for the function and $\boldsymbol{\phi}_1(\mathbf{x})$ for the values it produces at $\mathbf{x}$.

Finally, the output layer combines these m activations. We denote the network's output row by $\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x})$, where the subscript reminds us that the function depends on the chosen parameters. Its q coordinates are computed as

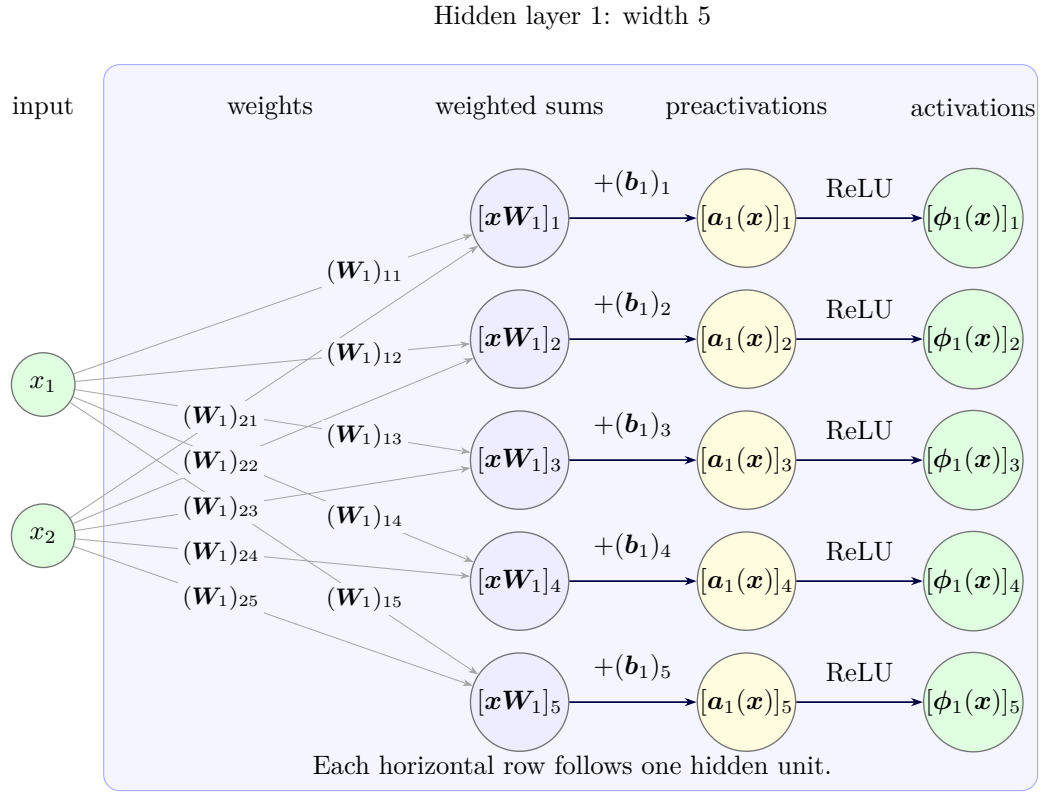
$$\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\phi}_1(\mathbf{x})\mathbf{W}_2 + \mathbf{b}_2. \quad (13)$$

The first two equations describe the hidden layer, while the third describes the output layer. The intermediate functions $\mathbf{a}_1$ and $\boldsymbol{\phi}_1$ depend on the hidden-layer parameters $\mathbf{W}_1$ and $\mathbf{b}_1$, although we omit $\boldsymbol{\theta}$ from their subscripts to keep the notation simple. Once the parameters are fixed, changing the input can still change the values computed by these functions.

To connect this notation with the earlier diagram, consider the case $d = 2$ and $m = 5$. This gives the calculation performed by the first hidden layer in Figure 2. Figure 3 shows its internal steps. In the node labels, square brackets followed by a subscript j indicate the j -th entry of the row inside the brackets. Thus $[\mathbf{a}_1(\mathbf{x})]_j$, for example, is the preactivation of hidden unit j .

In this expanded view, the circles represent intermediate values within a unit. Each horizontal path follows one unit through its weighted sum, the addition of its bias, and ReLU. The three internal columns show successive values computed by the same five units, which together form one hidden layer.

¹In the neural-network literature, hidden activations are often denoted using the letter h , which stands for *hidden*. Here we use ϕ to emphasise their role as features and connect this construction with the feature maps introduced in Lecture 2. This choice also avoids using the same letter for hidden activations and the classifiers denoted by h in Lecture 3.



The three columns show successive values within the same layer.

Figure 3: One hidden layer with two input coordinates and five units. The connection from input k to unit j carries the weight $(\mathbf{W}_1)_{kj}$. Each unit forms a weighted sum, adds its bias, and applies the scalar ReLU. The entries of $\mathbf{a}_1(\mathbf{x})$ and $\phi_1(\mathbf{x})$ are the preactivations and activations for the input shown.

Substituting the expressions for $\mathbf{a}_1(\mathbf{x})$ and $\phi_1(\mathbf{x})$ into the output formula gives a single expression for the whole network we considered in our example:

$$\mathbf{f}_\theta(\mathbf{x}) = \text{ReLU}(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2. \quad (14)$$

Unlike in the affine composition (8), we cannot in general combine the two weight matrices into a single matrix, because ReLU acts between them. This construction can therefore produce functions that are non-affine in the original input. Some parameter choices can still give an affine function. For example, if $\mathbf{W}_2 = \mathbf{0}$, the output is the constant row $\mathbf{b}_2$. Section 5 will give explicit examples of non-affine functions represented by networks.

Which features are learned? The hidden activations are the values of the features that the output layer combines. To see the dependence of one such feature on the input, we can write the j -th coordinate explicitly:

$$[\phi_1(\mathbf{x})]_j = \text{ReLU}\left(\sum_{k=1}^d x_k(\mathbf{W}_1)_{kj} + (\mathbf{b}_1)_j\right). \quad (15)$$

This is the same construction with which we began the section, now written for d input coordinates. If we hold $\mathbf{W}_1$ and $\mathbf{b}_1$ fixed, each coordinate of ϕ_1 is a fixed feature function. A fitting procedure that adjusts only $\mathbf{W}_2$ and $\mathbf{b}_2$ would therefore choose an affine combination of fixed features, as in Lecture 2.

If the fitting procedure also adjusts $\mathbf{W}_1$ and $\mathbf{b}_1$, however, it can change the feature functions themselves. It can therefore change both the features and how the output layer combines their values. This is the sense in which the network *learns features*. We choose the form of the construction through the architecture, while the fitted parameters determine the particular functions that it computes.

How the feature function changes. For one hidden neuron with two inputs, write its scalar feature as

$$\phi(\mathbf{x}) = \text{ReLU}(w_1x_1 + w_2x_2 + b).$$

If $(w_1, w_2) \neq (0, 0)$, the line $w_1x_1 + w_2x_2 + b = 0$ separates the region where the neuron is inactive (its feature value is zero) from the region where it is active (its feature value is positive). With the weights fixed, changing the bias translates this line. Changing the weights can change its orientation and how the feature varies on the active side. The ReLU function itself remains unchanged.

For example, changing (w_1, w_2, b) from $(1, -1, 0)$ to $(1, 1, -1)$ changes the feature from

$$\phi(\mathbf{x}) = \text{ReLU}(x_1 - x_2) \quad \text{to} \quad \tilde{\phi}(\mathbf{x}) = \text{ReLU}(x_1 + x_2 - 1). \quad (16)$$

Figure 4 shows both functions. At the same input $\mathbf{x} = (1, 2)$, their values are

$$\phi((1, 2)) = 0, \quad \tilde{\phi}((1, 2)) = 2.$$

The input stays the same, but the feature value changes because the parameters have changed. For d input coordinates and nonzero weights, the separating set is a hyperplane. In one dimension, it reduces to the breakpoint $x = -b/w$.

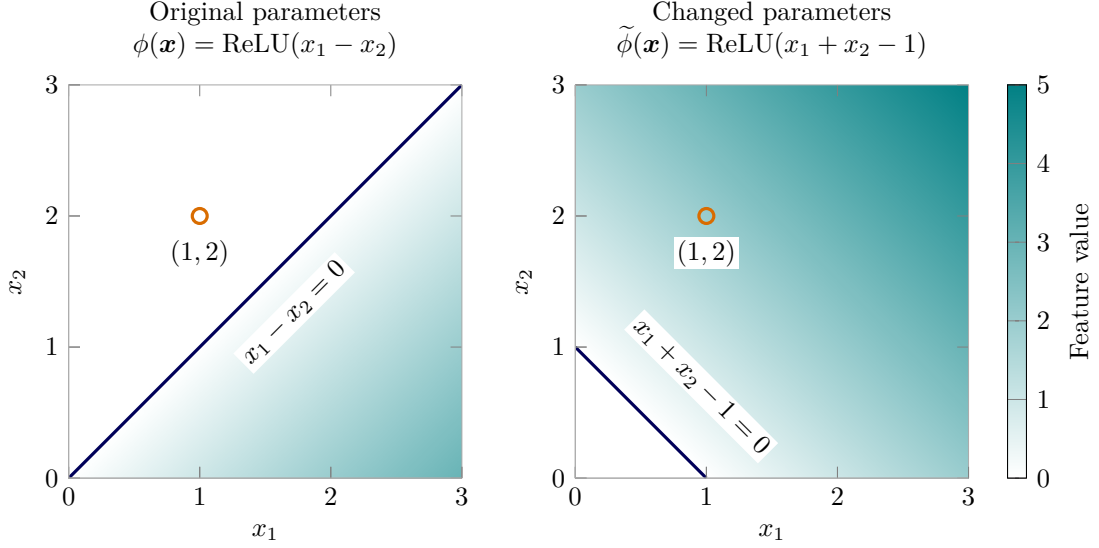


Figure 4: One feature before and after changing its weights and bias. Both panels use the same colour scale. White regions have feature value zero. The dark lines separate inactive and active regions; they are not classification boundaries. The orange circles mark the same input $(1, 2)$, whose feature value changes from zero to two.

3.4 A complete numerical forward pass

Let us now carry out a forward pass with actual numbers. For this example, we take two input coordinates, three hidden neurons, and one output, so $d = 2$, $m = 3$, and $q = 1$. With a single output, we identify the one-entry output row with a scalar and write its function as f_{θ} . Likewise, we write the output bias as a scalar b_2 . We choose the following parameter values:

$$\mathbf{W}_1 = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = (0 \quad 1 \quad 0), \quad \mathbf{W}_2 = \begin{pmatrix} 2 \\ -1 \\ 1/2 \end{pmatrix}, \quad b_2 = -1. \quad (17)$$

The three columns of $\mathbf{W}_1$ contain the weights for the three hidden units. Each column has two entries, one for each input coordinate. The matrix $\mathbf{W}_2$ has just one column, containing the three weights used by the sole output unit.

Consider the input row $\mathbf{x} = (1, 2)$. We first form a weighted sum using each column of $\mathbf{W}_1$, and add the corresponding entry of $\mathbf{b}_1$. This gives

$$\begin{aligned} [\mathbf{a}_1(\mathbf{x})]_1 &= 1 \cdot 1 + 2 \cdot (-1) + 0 = -1, \\ [\mathbf{a}_1(\mathbf{x})]_2 &= 1 \cdot 2 + 2 \cdot 0 + 1 = 3, \\ [\mathbf{a}_1(\mathbf{x})]_3 &= 1 \cdot (-1) + 2 \cdot 1 + 0 = 1. \end{aligned} \quad \mathbf{a}_1(\mathbf{x}) = \begin{pmatrix} -1 & 3 & 1 \end{pmatrix}. \quad (18)$$

The first preactivation is negative, so ReLU replaces it with zero. The other two are positive and remain unchanged. The row of hidden activations is therefore

$$\phi_1(\mathbf{x}) = \mathbf{ReLU}(\mathbf{a}_1(\mathbf{x})) = (0 \quad 3 \quad 1). \quad (19)$$

We now multiply these three values by the corresponding entries of $\mathbf{W}_2$, add the products, and add the output bias $b_2 = -1$. Writing s for the scalar output gives

$$s = f_{\theta}(\mathbf{x}) = 0 \cdot 2 + 3 \cdot (-1) + 1 \cdot \frac{1}{2} - 1 = -\frac{7}{2}. \quad (20)$$

Although the hidden activations are nonnegative, the output is negative. This is possible because the final weights and bias are unrestricted real numbers, and we have not applied ReLU after the output layer's affine transformation.

3.5 Several hidden layers

We can now extend the same calculation to networks with several hidden layers, such as the one in Figure 2. The input to each new hidden layer is the row of activations produced by the preceding layer. Consequently, we can describe the whole calculation by repeating the two steps that we used for the single hidden layer: an affine transformation followed by ReLU.

Let $L \geq 2$ be the total number of layers, including the hidden layers and the output layer. With our convention of excluding the input, this number is called the *depth* of the network. To allow different layers to have different widths, we let d_r denote the width of layer r . We also set $d_0 = d$ for the input dimension. Since the last layer produces q outputs, its width is $d_L = q$. The dimensions in their order through the network are therefore

$$d_0 = d, \quad d_1, \dots, d_{L-1}, \quad d_L = q.$$

Layer r receives d_{r-1} values and produces d_r values. It needs one weight for each connection from an incoming coordinate to one of its units, and one bias for each unit. Its parameters consequently have shapes

$$\mathbf{W}_r \in \mathbb{R}^{d_{r-1} \times d_r}, \quad \mathbf{b}_r \in \mathbb{R}^{1 \times d_r}, \quad r = 1, \dots, L.$$

As before, $\boldsymbol{\theta}$ collects all the adjustable entries, now across all L layers.

For the first hidden layer, the incoming row is $\mathbf{x}$. For each later hidden layer, it is the preceding row of hidden activations. Defining $\boldsymbol{\phi}_0(\mathbf{x}) = \mathbf{x}$ lets us write a single formula for both cases. Here the subscript zero simply refers to the input row.

For hidden layer r , we denote the row of preactivations by $\mathbf{a}_r(\mathbf{x})$ and the row of activations by $\boldsymbol{\phi}_r(\mathbf{x})$. We compute them in order, for $r = 1, \dots, L-1$, using

$$\begin{aligned} \mathbf{a}_r(\mathbf{x}) &= \boldsymbol{\phi}_{r-1}(\mathbf{x})\mathbf{W}_r + \mathbf{b}_r, \\ \boldsymbol{\phi}_r(\mathbf{x}) &= \text{ReLU}(\mathbf{a}_r(\mathbf{x})). \end{aligned} \tag{21}$$

For each r , the function $\boldsymbol{\phi}_r$ is therefore the feature map that takes the original input to the activations of hidden layer r . Once computed, the row $\boldsymbol{\phi}_r(\mathbf{x})$ provides the values needed by the next layer.

The output layer uses the final hidden row $\boldsymbol{\phi}_{L-1}(\mathbf{x})$ to form an affine combination, without applying ReLU. For consistent layer indexing, we denote this final affine row by $\mathbf{a}_L(\mathbf{x})$, which is also the output of the network:

$$\mathbf{a}_L(\mathbf{x}) = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\phi}_{L-1}(\mathbf{x})\mathbf{W}_L + \mathbf{b}_L. \tag{22}$$

How we interpret the output as a numerical prediction or a row of scores depends on the task. We will return to this distinction in Section 4.

Dimensions and parameter count. We can now return to the network in Figure 2. Listing its input dimension and layer widths gives the architecture

$$2 \longrightarrow 5 \longrightarrow 7 \longrightarrow 3. \tag{23}$$

Here $L = 3$, with $d_0 = 2$, $d_1 = 5$, $d_2 = 7$, and $d_3 = 3$. For example, the second layer receives five values and has seven units, so its weight matrix has five rows and seven columns, and its bias row has seven entries. Applying the same reasoning to each layer gives

Layer r	Weight matrix $\mathbf{W}_r$	Bias $\mathbf{b}_r$	Output shape
1	2×5	1×5	1×5
2	5×7	1×7	1×7
3	7×3	1×3	1×3

In general, layer r contains $d_{r-1}d_r$ weights and d_r biases. When all these entries are adjustable, adding their counts across the layers gives the total number of parameters, which we denote by P :

$$P = \sum_{r=1}^L (d_{r-1}d_r + d_r). \quad (24)$$

For the architecture (23), this gives

$$P = (2 \cdot 5 + 5) + (5 \cdot 7 + 7) + (7 \cdot 3 + 3) = 15 + 42 + 24 = 81.$$

4 What does the output mean?

We now know how to compute the output of a network. However, this calculation alone does not specify how we will use that output. In regression, we will use it directly as a numerical prediction. In classification, instead, we will interpret the outputs as scores and convert them into probabilities using the sigmoid or softmax, as in Lecture 3.

We will then compare the predictions with the observed targets using the loss functions introduced in Lectures 2 and 3: squared loss for regression, logistic loss for binary classification, and cross-entropy loss for multiclass classification. We will now describe how the network's output enters each of these constructions.

4.1 Regression

For the regression problems considered here, each target is a single real number. We therefore choose an output layer with one unit, so $d_L = 1$. For an input $\mathbf{x}$, we use the scalar output $f_{\boldsymbol{\theta}}(\mathbf{x})$ directly as the numerical prediction.

If the observed target is y , we evaluate this prediction using the squared loss from Lecture 2:

$$\ell_{\text{sq}}(f_{\boldsymbol{\theta}}(\mathbf{x}), y) = (f_{\boldsymbol{\theta}}(\mathbf{x}) - y)^2. \quad (25)$$

For a dataset, we apply the same network to each input and add the individual losses. This gives the empirical squared loss

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \sum_{i=1}^N (f_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i)^2. \quad (26)$$

4.2 Binary classification

For binary classification with labels $y \in \{0, 1\}$, we also choose an output layer with one unit, so $d_L = 1$. For an input $\mathbf{x}$, however, we interpret the scalar output $f_{\boldsymbol{\theta}}(\mathbf{x})$ as a score, also called

a *logit*. As in Lecture 3, we apply the sigmoid to obtain the probability assigned to class 1:

$$p_{\theta}(\mathbf{x}) = \sigma(f_{\theta}(\mathbf{x})). \quad (27)$$

If the observed label is y , we evaluate this probability using the logistic loss from Lecture 3, $\ell_{\log}(p_{\theta}(\mathbf{x}), y)$. For a dataset, we repeat this calculation for each input and add the individual losses. This gives the empirical logistic loss

$$\mathcal{L}_{\log}(\theta) = \sum_{i=1}^N \ell_{\log}(p_{\theta}(\mathbf{x}_i), y_i). \quad (28)$$

Finally, we use the plug-in rule from Lecture 3 to convert the probability into a predicted class:

$$h_{p_{\theta}}(\mathbf{x}) = \text{step}\left(p_{\theta}(\mathbf{x}) - \frac{1}{2}\right) = \text{step}(f_{\theta}(\mathbf{x})). \quad (29)$$

Thus we predict class 1 when the score is positive and class 0 otherwise. A score of zero gives class 0.

4.3 Multiclass classification

For K classes, labelled $1, \dots, K$, we use one score per class. We therefore choose an output layer with K units, so $d_L = K$. For an input $\mathbf{x}$, we interpret the row of outputs $\mathbf{f}_{\theta}(\mathbf{x})$ as class scores, also called *logits*. As in Lecture 3, we apply softmax to obtain the row of class probabilities:

$$\mathbf{p}_{\theta}(\mathbf{x}) = \text{softmax}(\mathbf{f}_{\theta}(\mathbf{x})). \quad (30)$$

Its k -th entry, $p_{\theta,k}(\mathbf{x})$, is the probability assigned to class k .

If the observed label is y , we evaluate these probabilities using the cross-entropy loss from Lecture 3, $\ell_{\text{multi}}(\mathbf{p}_{\theta}(\mathbf{x}), y)$. For a dataset, we repeat this calculation for each input and add the individual losses. This gives the empirical cross-entropy loss

$$\mathcal{L}_{\text{multi}}(\theta) = \sum_{i=1}^N \ell_{\text{multi}}(\mathbf{p}_{\theta}(\mathbf{x}_i), y_i). \quad (31)$$

Finally, we use the plug-in rule from Lecture 3 to convert the probabilities into a predicted class. We select a class with the largest probability, choosing the smallest index in a tie:

$$h_{\mathbf{p}_{\theta}}(\mathbf{x}) = \min \left(\underset{k \in \{1, \dots, K\}}{\text{argmax}} \ p_{\theta,k}(\mathbf{x}) \right). \quad (32)$$

5 What the hidden layers can represent

We have seen how a network computes its output and how to use that output in a learning problem. We can now ask what the hidden transformations make possible. We will answer this with two small constructions. The first produces a triangular function of one variable. The second separates four points that no affine score can classify correctly.

5.1 Three ReLUs can form a triangular function

Suppose that we want a tent shaped function which is zero up to $x = 0$, rises to height one at $x = 1$, falls back to zero at $x = 2$, and then remains zero. We can build it by changing the slope at these three points.

The first term, $\text{ReLU}(x)$, starts the rise at zero with slope one. However, it would continue rising after $x = 1$. To make the slope change from one to minus one there, we subtract $2\text{ReLU}(x - 1)$. Finally, at $x = 2$, adding $\text{ReLU}(x - 2)$ changes the slope from minus one back to zero. This gives

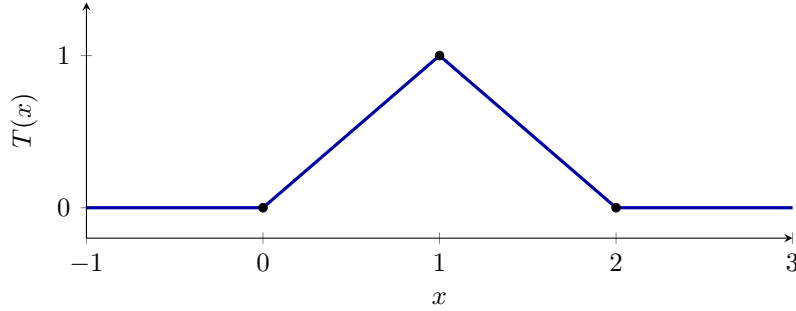
$$T(x) = \text{ReLU}(x) - 2\text{ReLU}(x - 1) + \text{ReLU}(x - 2). \quad (33)$$

We can check the construction by considering which ReLUs are active on each interval:

Interval	Calculation	Result
$x \leq 0$	$0 - 0 + 0$	0
$0 < x \leq 1$	$x - 0 + 0$	x
$1 < x < 2$	$x - 2(x - 1) + 0$	$2 - x$
$x \geq 2$	$x - 2(x - 1) + (x - 2)$	0

At each breakpoint, the adjacent expressions agree. Therefore,

$$T(x) = \begin{cases} 0, & x \leq 0, \\ x, & 0 < x \leq 1, \\ 2 - x, & 1 < x < 2, \\ 0, & x \geq 2. \end{cases} \quad (34)$$



This is exactly a network with one input, three hidden neurons and one output. Its parameters are

$$\mathbf{W}_1 = \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 & -1 & -2 \end{pmatrix}, \quad \mathbf{W}_2 = \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}, \quad b_2 = 0. \quad (35)$$

The first layer produces the features $\text{ReLU}(x)$, $\text{ReLU}(x - 1)$ and $\text{ReLU}(x - 2)$. The second combines them with coefficients $1, -2, 1$. In this construction, the hidden biases locate the breakpoints and the final weights determine the changes of slope.

Could a single affine function produce the same triangle? Suppose that $ax + b$ did so. Since $T(0) = 0$, we would need $b = 0$. Since $T(2) = 0$, we would then need $2a = 0$, so $a = 0$. However, this would give value zero at $x = 1$, whereas $T(1) = 1$. Consequently, the triangular function cannot be represented by one affine layer.

The graph shows that T is both *continuous* and *piecewise affine*: its domain can be divided into intervals on each of which it has an affine formula, and adjacent pieces meet without jumps.

For any fixed choice of parameters, a finite ReLU network with affine output has the same two properties. Within a region where the pattern of active and inactive neurons stays fixed, each ReLU either passes its input through or replaces it by zero. The network therefore computes a composition of affine maps, which is itself affine. Since there are only finitely many activation patterns, the network's output is piecewise affine as a function of its input. Moreover, affine maps and ReLU are continuous, so their composition is continuous: the affine pieces meet without jumps.

In classification, we remark that it is the network's scores that are continuous and piecewise affine functions of the input: applying sigmoid or softmax preserves continuity, but the resulting class probabilities are generally no longer piecewise affine.

5.2 A network that separates XOR

For our classification example, consider four inputs with labels

$$\begin{array}{c|cccc} \mathbf{x} & (0,0) & (1,0) & (0,1) & (1,1) \\ \hline y & 0 & 1 & 1 & 0. \end{array} \quad (36)$$

This labelling is called *exclusive or*, or *XOR*: the label is one when exactly one of the two coordinates is one.

We first check why an affine score cannot classify all four points correctly. Collect the parameters of the affine predictor in

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ w_1 \\ w_2 \end{pmatrix}$$

so that, as in Lecture 2, we can write the score as

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + b, \quad \mathbf{x} = (x_1, x_2).$$

Our plug-in rule predicts class 0 for a nonpositive score and class 1 for a positive score. Therefore, classifying all four inputs correctly would require

$$\begin{aligned} b &= w_1 \cdot 0 + w_2 \cdot 0 + b = f_{\boldsymbol{\theta}}(0,0) \leq 0, \\ w_1 + w_2 + b &= w_1 \cdot 1 + w_2 \cdot 1 + b = f_{\boldsymbol{\theta}}(1,1) \leq 0, \\ w_1 + b &= w_1 \cdot 1 + w_2 \cdot 0 + b = f_{\boldsymbol{\theta}}(1,0) > 0, \\ w_2 + b &= w_1 \cdot 0 + w_2 \cdot 1 + b = f_{\boldsymbol{\theta}}(0,1) > 0. \end{aligned} \quad (37)$$

The first two inequalities correspond to the inputs with label 0. Adding them gives

$$w_1 + w_2 + 2b \leq 0.$$

However, the last two inequalities correspond to the inputs with label 1, and adding them gives

$$w_1 + w_2 + 2b > 0.$$

The same number cannot satisfy both requirements. Therefore, no choice of $\boldsymbol{\theta}$ allows this affine score to classify all four inputs correctly using our plug-in rule.

We now construct a network whose scores allow the plug-in rule to classify all four inputs correctly. To do this, notice that the two coordinates are different at the inputs with label 1, whereas they are equal at the inputs with label 0. The feature $\text{ReLU}(x_1 - x_2)$ detects a positive difference in one direction, and $\text{ReLU}(x_2 - x_1)$ detects a positive difference in the other. Their sum is zero when the coordinates agree and one at either of our class-1 inputs. Therefore, subtracting $1/2$ gives the score

$$f_{\theta}(\mathbf{x}) = \text{ReLU}(x_1 - x_2) + \text{ReLU}(x_2 - x_1) - \frac{1}{2}. \quad (38)$$

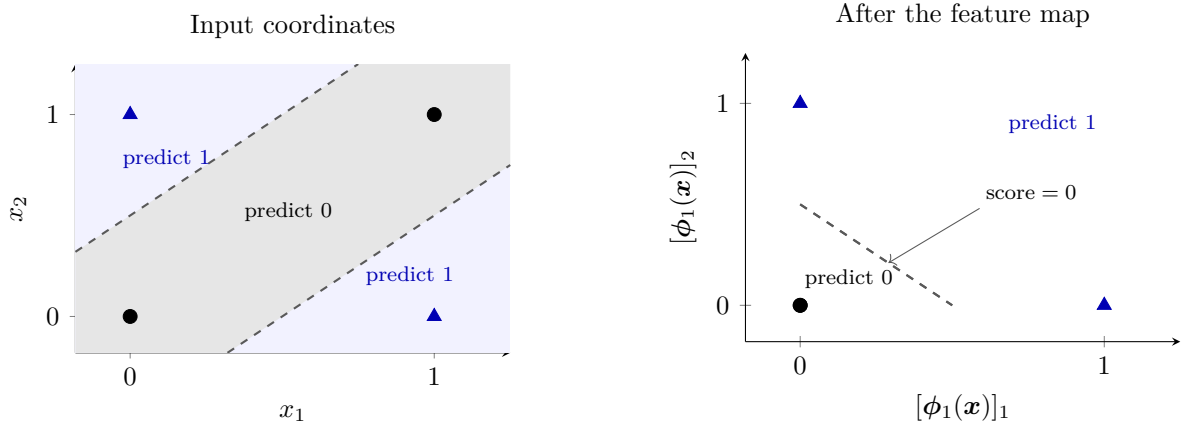
This uses two hidden neurons, with parameters

$$\mathbf{W}_1 = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 & 0 \end{pmatrix}, \quad \mathbf{W}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad b_2 = -\frac{1}{2}. \quad (39)$$

Evaluating the hidden features and then the final score gives

Input	$\text{ReLU}(x_1 - x_2)$	$\text{ReLU}(x_2 - x_1)$	Score	Predicted label
(0, 0)	0	0	$-1/2$	0
(1, 0)	1	0	$1/2$	1
(0, 1)	0	1	$1/2$	1
(1, 1)	0	0	$-1/2$	0

All four predictions are correct. Moreover, since $\text{ReLU}(u) + \text{ReLU}(-u) = |u|$, we see that the score $f_{\theta}(\mathbf{x})$ has the equivalent expression $f_{\theta}(\mathbf{x}) = |x_1 - x_2| - 1/2$.



Circles have label 0 and triangles have label 1. On the left, the network predicts class 0 in the central band, where $|x_1 - x_2| \leq 1/2$, and class 1 outside it. The dashed lines are where $|x_1 - x_2| = 1/2$, so the score is zero. Points on these lines are assigned to class 0.

On the right, each input has been replaced by its feature row $\phi_1(\mathbf{x})$, while keeping its original label. The inputs (0, 0) and (1, 1) both map to (0, 0), so their circles overlap. The other two inputs map to (1, 0) and (0, 1). The final layer adds the two feature values and subtracts $1/2$, so its score is zero on the straight line where those values sum to $1/2$. Points on this line are assigned to class 0. The line separates the circle from both triangles, so the transformed dataset is linearly separable.

Although all labels are correct, the probabilities are $\sigma(-1/2)$ and $\sigma(1/2)$, rather than zero and one. The logistic loss is therefore still positive. Multiplying the whole score by a positive number leaves these four predicted labels unchanged but changes their probabilities. Exercise 8 explores this distinction.

5.3 What these constructions show

Both examples work by transforming the input before applying an affine output layer. They do not establish that a neural network is the only way to obtain the desired predictions. Indeed, we could fix the three ReLU features used for the triangle and fit only their final coefficients. For XOR, adding the fixed feature x_1x_2 would also suffice: the score $x_1 + x_2 - 2x_1x_2 - 1/2$ has the required signs at our four binary inputs.

The connection with learned features is therefore essential: with a fixed feature expansion—as we did in Lecture 2—we decide the feature functions $\phi_1, \dots, \phi_m$ before fitting their coefficients. In a network, the transformations also have parameters, so fitting all layers can adjust the features together with their final combination. For example, adjusting a hidden bias can move the point at which a ReLU changes slope. We still choose the architecture and activation function in advance, but we allow the data to guide the choice of transformations within that family.

In the two examples above, we supplied the parameters ourselves. Consequently, we have shown that suitable parameters exist for these particular tasks, rather than that a training procedure (such as gradient descent) will find them.

As we already briefly mentioned in Lectures 2 and 3, fitting the observed data well does not necessarily mean predicting accurately on *new* observations. Again, the ability to predict well on *unseen* data is called *generalisation*, and we stress that a small training loss alone does not guarantee good generalisation, with possible obstacles given by *underfitting*, when the model fails to capture relevant patterns in the data, and *overfitting*, when it fits details of the training data that do not carry over to new observations.

In this lecture and the next, we focus on the fitting procedure, which uses derivatives of the loss to adjust the network's parameters.

6 Fitting the network: stationary points and gradient descent

For the affine models we built in Lectures 2 and 3, we proved that every stationary point of the squared loss or the logistic loss is a global minimiser. With neural networks, this guarantee no longer holds: a stationary point need not be a global minimiser. Furthermore, we will see that gradient descent can decrease the loss at every step and still converge to a local minimum whose loss is larger than the smallest value the model can achieve.

6.1 Derivatives and parameter updates

To fit the parameters of the network, we seek values of its weights and biases that minimise the empirical loss associated with the dataset and the task under consideration. Where this loss is differentiable, we can try to minimise it using the same gradient-descent algorithm as in linear regression and linear classification. Writing $\mathcal{L}$ for any of the empirical losses chosen in Section 4, the update is

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}), \quad \eta > 0. \quad (40)$$

Here t counts updates, and each component of the gradient is a partial derivative with respect to one weight or bias.

To compute these derivatives, we shall apply the chain rule through the network. In particular, we need the derivative of ReLU. Since ReLU is constant for negative inputs and is the identity

for positive inputs,

$$\text{ReLU}'(a) = \begin{cases} 0, & a < 0, \\ 1, & a > 0, \end{cases} = \text{step}(a), \quad a \neq 0. \quad (41)$$

Thus its derivative agrees with the Heaviside step function away from zero. On the other hand, at zero, the left derivative is zero and the right derivative is one, so ReLU is *not* differentiable there. For the parameter updates, we adopt the following computational convention: we use $\text{step}(a)$ as the multiplier associated with ReLU, including $\text{step}(0) = 0$, as PyTorch does. This convention does not make ReLU differentiable at zero: its ordinary derivative remains undefined there.

6.2 A network with one adjustable parameter

To see what can go wrong during optimisation, we will use a network whose loss depends on just one scalar parameter θ . This will allow us to plot the entire objective. Consider

$$f_\theta(x) = \text{ReLU}(x - \theta) + \text{ReLU}(\theta - x) - 3 = |x - \theta| - 3. \quad (42)$$

The two ReLUs are hidden units, and the output adds their activations and subtracts three. We keep all other coefficients fixed and adjust only θ . In particular, the two hidden biases are $-\theta$ and θ , so they change together. Changing θ moves the vertex of the V-shaped graph horizontally: its coordinates are $(\theta, -3)$, as illustrated in Figure 5.

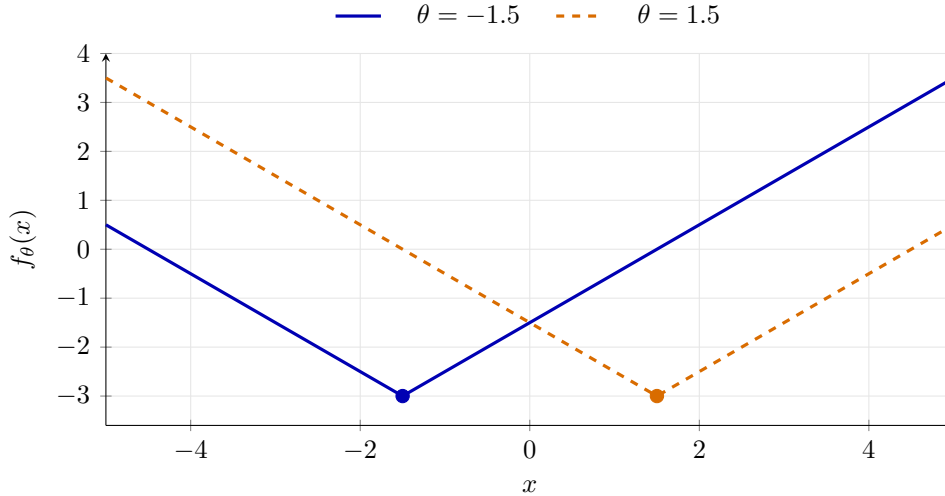


Figure 5: The network output $f_\theta(x) = |x - \theta| - 3$ as a function of the input x , for two choices of θ . Changing θ translates the graph horizontally without changing its shape. The marked vertices have coordinates $(\theta, -3)$.

We will use the inputs -1 and 1 in both examples, changing the targets and the loss to suit the prediction task. We will distinguish a *global minimum*, which gives the smallest loss over all choices of θ , from a *local minimum*, where no sufficiently small change in θ reduces the loss. A local minimum can therefore have a larger loss than another parameter choice farther away.

6.3 Regression with squared loss

Take the two observations

$$(x_1, y_1) = (-1, -1), \quad (x_2, y_2) = (1, 1).$$

The network outputs are numerical predictions, so the objective is

$$\mathcal{L}_{\text{sq}}(\theta) = (f_{\theta}(-1) + 1)^2 + (f_{\theta}(1) - 1)^2. \quad (43)$$

Substituting (42) gives a piecewise quadratic function. The expression changes when θ passes either input, because one of the absolute values changes its formula:

$$\mathcal{L}_{\text{sq}}(\theta) = \begin{cases} 2(\theta + 3)^2, & \theta \leq -1, \\ 2(\theta + 1)^2 + 8, & -1 < \theta < 1, \\ 2(\theta - 3)^2 + 8, & \theta \geq 1. \end{cases} \quad (44)$$

The first expression is minimised at $\theta = -3$, where the loss is zero. Indeed, the two predictions are then -1 and 1 , exactly matching the targets. Since the squared loss cannot be negative, this is a global minimum.

However, the third expression also has a minimum, at $\theta = 3$, where the loss is eight. The predictions are now 1 and -1 , so their errors are 2 and -2 . A sufficiently small change in θ increases the term $2(\theta - 3)^2$, making this a local minimum. Both points are stationary:

$$\mathcal{L}'_{\text{sq}}(-3) = \mathcal{L}'_{\text{sq}}(3) = 0.$$

Nevertheless, their losses are different. Solving the stationarity equation can therefore give us a local minimum whose loss is larger than the global minimum.

The left panel of Figure 6 shows the objective. The horizontal axis varies the parameter θ , while the observations remain fixed. At each marked minimum the loss is differentiable and its derivative is zero, but the two minima have different loss values.

6.4 Binary classification with logistic loss

Now use the same inputs with class labels zero and one:

$$(x_1, y_1) = (-1, 0), \quad (x_2, y_2) = (1, 1).$$

The network outputs are scores, and $p_{\theta}(x) = \sigma(f_{\theta}(x))$ is the probability assigned to class 1. The objective is therefore

$$\mathcal{L}_{\log}(\theta) = -\log(1 - p_{\theta}(-1)) - \log p_{\theta}(1). \quad (45)$$

As in Lecture 3, the derivatives of the two individual losses with respect to their scores are $p_{\theta}(-1)$ and $p_{\theta}(1) - 1$, respectively. We multiply each of these by the derivative of its score with respect to θ and add the results. This gives

$$\mathcal{L}'_{\log}(\theta) = \begin{cases} -\sigma(-\theta - 4) + \sigma(\theta + 2), & \theta < -1, \\ \sigma(\theta - 2) + \sigma(\theta + 2), & -1 < \theta < 1, \\ \sigma(\theta - 2) - \sigma(4 - \theta), & \theta > 1. \end{cases} \quad (46)$$

On each outer interval, the derivative is increasing and changes from negative to positive: at $\theta = -3$ on the left and at $\theta = 3$ on the right. Both points are therefore local minima. On the middle interval the derivative is positive, so the loss is increasing there. Comparing the losses at the two minima will therefore identify the global one.

At $\theta = -3$, the scores are -1 and 1 , giving

$$p_{-3}(-1) = \sigma(-1) \approx 0.269, \quad p_{-3}(1) = \sigma(1) \approx 0.731.$$

Both predicted classes are correct, and each observation assigns probability $1/(1 + e^{-1})$ to its observed class. Consequently,

$$\mathcal{L}_{\log}(-3) = 2 \log(1 + e^{-1}) \approx 0.627.$$

At $\theta = 3$, however, the scores are reversed:

$$p_3(-1) = \sigma(1) \approx 0.731, \quad p_3(1) = \sigma(-1) \approx 0.269.$$

Both predicted classes are now wrong, and the loss is

$$\mathcal{L}_{\log}(3) = 2 \log(1 + e) \approx 2.627.$$

Thus $\theta = -3$ is the global minimum, while $\theta = 3$ is a local minimum with a larger loss. In particular, the derivative is zero at a point where the classifier makes a mistake on every observation.

The right panel of Figure 6 shows the objective. The horizontal axis varies the parameter θ , while the observations remain fixed. At each marked minimum the loss is differentiable and its derivative is zero, but the two minima have different loss values.

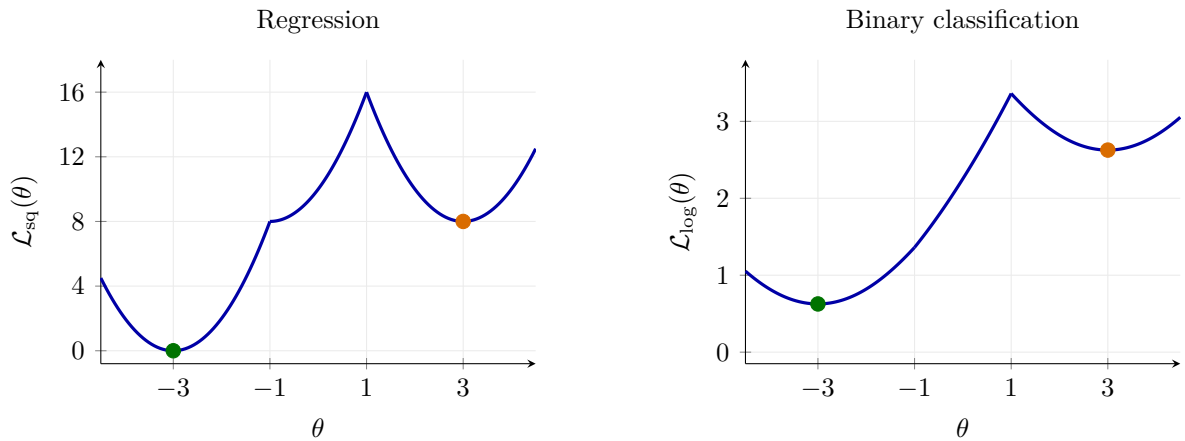


Figure 6: The two losses as functions of the single adjustable parameter. The green point on each curve is the global minimum at $\theta = -3$. The orange point is a strict local minimum at $\theta = 3$, where the loss is larger. Both losses are differentiable at these points, and their ordinary derivatives are zero there.

6.5 Where gradient descent can stop

The two examples above also illustrate what can go wrong when we try to minimise the empirical loss using an iterative procedure such as gradient descent.

With one parameter, gradient descent takes the scalar form

$$\theta^{(t+1)} = \theta^{(t)} - \eta \mathcal{L}'(\theta^{(t)}). \quad (47)$$

For either objective above, choose $\eta = 0.2$. Starting from $\theta^{(0)} = -4$, the updates converge to the global minimum at -3 . Starting from $\theta^{(0)} = 4$, instead, they converge to the local minimum at 3 , as shown in Figure 7. Both runs decrease their loss throughout the updates, but they reach different limiting values. Decreasing the loss at every step therefore does not guarantee convergence to its global minimum.

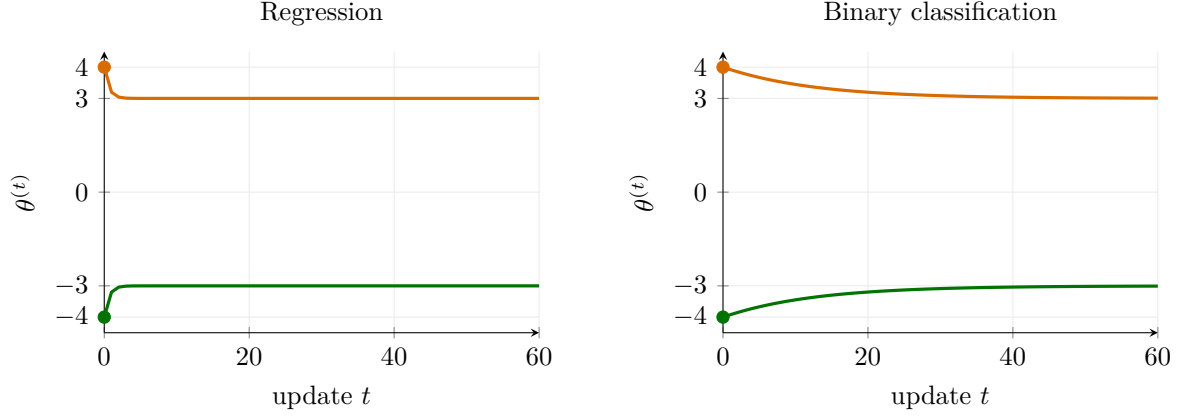


Figure 7: Gradient-descent iterates with learning rate $\eta = 0.2$. For each loss, the run starting from $\theta^{(0)} = -4$, shown in green, approaches the global minimum at -3 . The run starting from $\theta^{(0)} = 4$, shown in orange, approaches the local minimum at 3 , where the loss is larger.

7 Finding useful solutions in practice

Our examples show that gradient descent can converge to a poor local minimum even when the loss decreases at every step. In practice, several strategies can help address this problem.

- **Try different initialisations.** We can repeat training from different randomly chosen parameter values. Different starting points can lead to different final solutions, so comparing their training losses can reveal whether one run found a better fit than another. Repeating the procedure increases the computational cost and does not guarantee that any run reaches a global minimum.

For the losses above, starting from $\theta^{(0)} = -1.5$ or $\theta^{(0)} = 1.5$ leads to different minima. Figure 8 shows the updates directly on each loss curve: both runs move downhill, but reach different loss values.

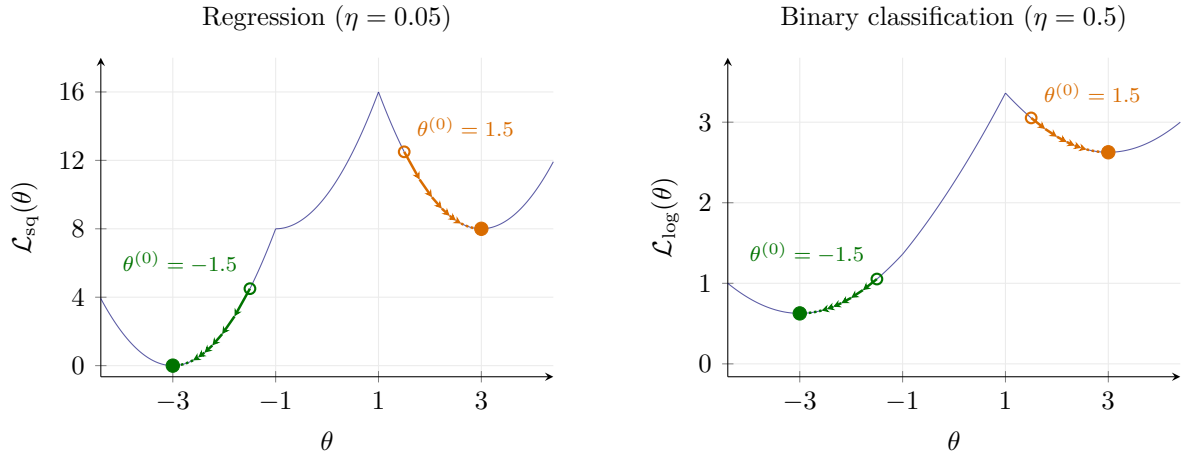


Figure 8: Gradient descent from two initial values on the loss curves of Figure 6. Open circles mark the starting points; arrows connect the first six consecutive updates. Dotted paths show later updates towards the limiting minima, marked by filled circles. For each loss, starting from $\theta^{(0)} = -1.5$ leads to the global minimum at -3 , while starting from $\theta^{(0)} = 1.5$ leads to the suboptimal local minimum at 3 . Within each panel, both runs use the same learning rate, given in the title.

- **Change how the updates are calculated.** Mini-batch stochastic gradient descent (SGD) computes each update using a small, randomly selected group of observations.

At a local minimum of the empirical loss, the gradients contributed by individual observations may be nonzero but cancel when added together. This cancellation need not occur within a randomly selected mini-batch, so SGD may continue to move where gradient descent using all observations would stop.

The resulting random fluctuations can sometimes help the iterates leave the region around a suboptimal local minimum. However, a nonzero mini-batch gradient alone does not guarantee this, nor does SGD guarantee convergence to a global minimum. For example, SGD may start at a point that is not a global minimiser but where the loss for each individual observation has zero gradient. Every mini-batch gradient is then also zero, so SGD makes no update.

- **Consider a different architecture.** Additional hidden neurons provide more adjustable ways to change the predictions. A wider network can sometimes be easier to optimise, as well as more expressive. However, adding parameters does not automatically remove poor local minima or make every initialisation successful.

There are theoretical conditions under which the strategies described above can help optimisation, and in certain settings convergence to a global minimum can be proved. However, these results rely on specific assumptions: a general theory explaining when these strategies succeed across the range of networks and datasets used in practice is still lacking. Outside the settings covered by such results, we therefore use these approaches as *heuristic strategies*: their effectiveness must be assessed experimentally, rather than assumed to follow from a theoretical guarantee.

8 Computing the derivatives in the laboratory

For a network with many adjustable weights and biases, computing the gradient of the empirical loss requires a partial derivative with respect to each parameter. In the laboratory, PyTorch's *automatic differentiation* will compute these derivatives for us. We specify which parameters require derivatives, calculate the network outputs and the loss, and ask PyTorch to differentiate that loss. We then use the resulting derivatives to update the parameters.

In Lecture 5, we will study *backpropagation*, the algorithm behind these gradient computations in PyTorch. It organises the calculation by applying the chain rule and reusing intermediate results.

9 Exercises

The exercises begin with the dimensions of a network and the effect of composing its layers. We then practise a complete forward calculation and distinguish fixed features from features whose parameters are learned. We check why ReLU is not differentiable at zero and examine how initialisation affects gradient descent. The final two exercises construct a function using ReLU and distinguish correct class predictions from the probabilities assigned to those classes.

1. Dimensions and parameters. A network takes an input row $\mathbf{x} \in \mathbb{R}^{1 \times 3}$, has one ReLU hidden layer of width 4, and outputs two logits. Both affine layers have biases.

- (a) State the shapes of both weight matrices, both biases, and the hidden preactivation and activation rows. Write $[\mathbf{a}_1(\mathbf{x})]_j$ in terms of the input coordinates, first-layer weights and bias.
- (b) Count the scalar parameters. How many would a single affine map from the same inputs to the same outputs have?

2. Two affine layers. Let

$$\mathbf{u}(\mathbf{x}) = \mathbf{x} \begin{pmatrix} 1 & 0 \\ 2 & -1 \end{pmatrix} + \begin{pmatrix} 1 & -1 \end{pmatrix}, \quad v(\mathbf{z}) = \mathbf{z} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + 3. \quad (48)$$

- (a) Compute the effective weight column and bias of $v \circ \mathbf{u}$. Evaluate the composition at $\mathbf{x} = (0, 1)$, both by the two stages and by the collapsed formula.
- (b) Now insert ReLU between the stages and define $F(\mathbf{x}) = v(\text{ReLU}(\mathbf{u}(\mathbf{x})))$. Evaluate F at the same input.
- (c) To check whether F is affine, consider inputs $\mathbf{x} = (t, 0)$. Write $F((t, 0))$ as a piecewise-defined function of t . Use its slopes to determine whether F is affine.

3. A new input to the worked network. Use the parameters (17) with $\mathbf{x} = (2, 0)$.

- (a) Compute the hidden preactivations, hidden activations and scalar output.
- (b) Interpret the output as a regression prediction with target $y = 1$, and compute the squared loss.
- (c) Instead interpret it as a binary logit with target $y = 1$. Give the class-1 probability, predicted label and logistic loss.

4. Fixed features and learned features. Return to the same worked network, whose first-layer parameters are

$$\mathbf{W}_1 = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 & 1 & 0 \end{pmatrix}.$$

Consider the two inputs $\mathbf{x}_A = (-1, -1)$ and $\mathbf{x}_B = (-2, -2)$. We want regression outputs 1 and 0, respectively.

- (a) Write the hidden feature row $\phi_1(\mathbf{x})$ as a function of a general input $\mathbf{x} = (x_1, x_2)$, and evaluate it at $\mathbf{x}_A$ and $\mathbf{x}_B$.
- (b) Keep $\mathbf{W}_1$ and $\mathbf{b}_1$ fixed. Can any choice of final-layer weights $\mathbf{W}_2$ and bias b_2 produce the two required outputs? Justify your answer for all choices of these final-layer parameters.
- (c) Change the second entry of $\mathbf{b}_1$ from 1 to 3, keeping $\mathbf{W}_1$ and the other hidden biases unchanged. Denote the new hidden feature map by $\hat{\phi}_1$. Evaluate it at both inputs and find one choice of $\mathbf{W}_2$ and b_2 that now produces the required outputs. Verify both predictions.

5. ReLU at zero. Compute the left and right derivatives of ReLU at zero using difference quotients, and determine whether an ordinary derivative exists there.

6. Gradient descent from two initialisations. Return to the regression problem for the network (42). Use the squared-loss formula in (44).

- (a) Calculate $\mathcal{L}'_{\text{sq}}(\theta)$ at $\theta = -4, 0, 4$. For each point, use the sign of the derivative to state whether a gradient-descent update with $\eta > 0$ increases or decreases θ .
- (b) For the same regression problem, perform two gradient-descent updates with $\eta = 0.2$, first starting from $\theta^{(0)} = -2$ and then starting from $\theta^{(0)} = 2$. In each run, compute the squared loss at $t = 0, 1, 2$. Use the update formula in each region to justify which minimum each run approaches. Give the limiting loss of each run, and use these two runs to test the claim that decreasing the loss at every update guarantees convergence to a global minimum.

7. Constructing a function with ReLU.

- (a) Write

$$U(x) = 2\text{ReLU}(x + 1) - 4\text{ReLU}(x) + 2\text{ReLU}(x - 1) \quad (49)$$

as a piecewise-defined function, and find its value at $x = 0$.

- (b) Give weights and biases of a one-hidden-layer network that represents U exactly.

8. Correct labels, probabilities and loss. Return to the four XOR observations and the network in (38).

- (a) Keep the first layer fixed and give final-layer weights and bias that multiply the original score by a factor $\alpha > 0$. Denote the resulting parameter vector by $\boldsymbol{\theta}(\alpha)$. Compute the four scores, the predicted labels and the probability assigned to each observed label.
- (b) Calculate the logistic loss of each observation and hence the total loss $\mathcal{L}_{\text{log}}(\boldsymbol{\theta}(\alpha))$. Determine whether the total loss can be zero for any finite $\alpha > 0$, and calculate its limit as α tends to infinity.